pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

From Notebook to Module

WEEK 4 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note

The EDA notebook's cleaning routine lives in one notebook, and October's extract arrives next month. Put it and its checks in one file anyone on the desk can import. Run it on September and tie the cover note. Count what it changes, and prove it leaves a file with no problems alone. Write me the workbook. Test the function itself, so an edit next year cannot break it quietly. And tell me which of our other spreadsheet jobs deserve the same.

A module is a plain text file ending in ".py" that holds functions; a notebook imports it and calls them.

September file	What it is
`holdings_messy.csv`	the extract as operations sent it, the EDA notebook's file
`holdings.csv`	the same book, sent with no problems

The issuers are invented; every statement describes the data.

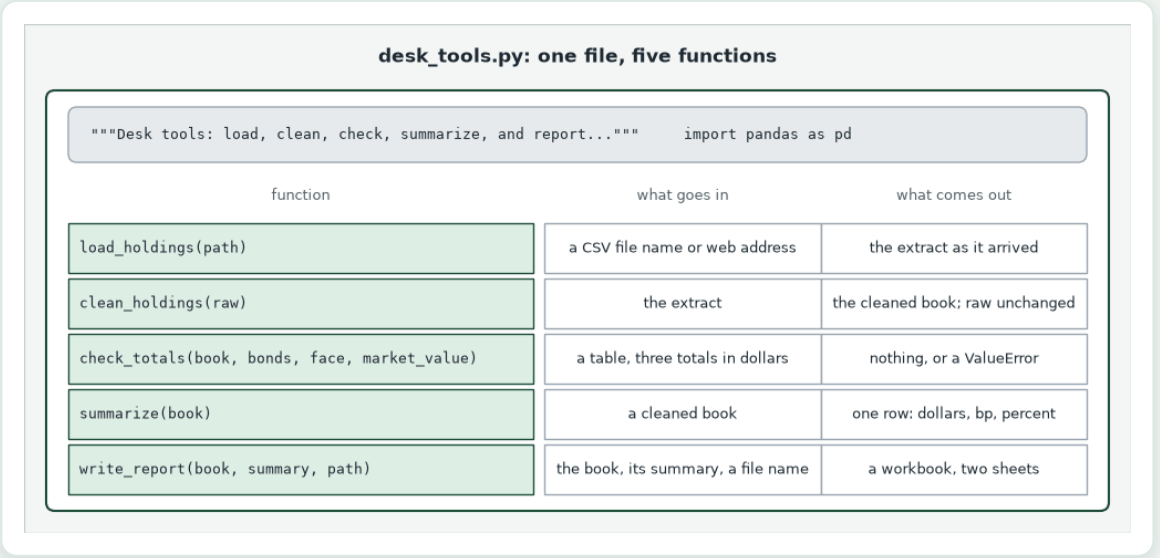
One file holds the routine; the notebook only calls it.

19.1 The routine in a file

```
expected_bonds = 200
expected_face = 489_000_000
expected_value = 490_819_215
print(os.path.exists('desk_tools.py'))
False
```

A cell that starts "%%writefile desk_tools.py" runs nothing: Jupyter saves the rest of the cell as that file. One line, "load_holdings"'s first, carries notes on the types going in and out; 19.2 reads them.

```
%%writefile desk_tools.py
import pandas as pd
def load_holdings(path: str) -> pd.DataFrame:
def clean_holdings(raw):
def check_totals(book, bonds, face, market_value):
def summarize(book):
def write_report(book, summary, path):
Writing desk_tools.py
```



Import it, run September, and look inside

```
import desk_tools
raw = desk_tools.load_holdings(data_folder + 'holdings_messy.csv')
book = desk_tools.clean_holdings(raw)
desk_tools.check_totals(book, expected_bonds, expected_face, expected_value)
print(f"{len(raw)} rows in, {len(book)} out, {book['market_value'].sum():,.2f} dollars")
204 rows in, 200 out, 490,819,215.00 dollars
```

"check_totals" raised nothing: all three totals tie. Unlike the EDA notebook's "clean_holdings(raw, ratings)", this one needs only the extract: no rating scale, no date conversion.

```
print([name for name in dir(desk_tools) if not name.startswith('_')])
['check_totals', 'clean_holdings', 'load_holdings', 'pd', 'summarize', 'write_report']
```

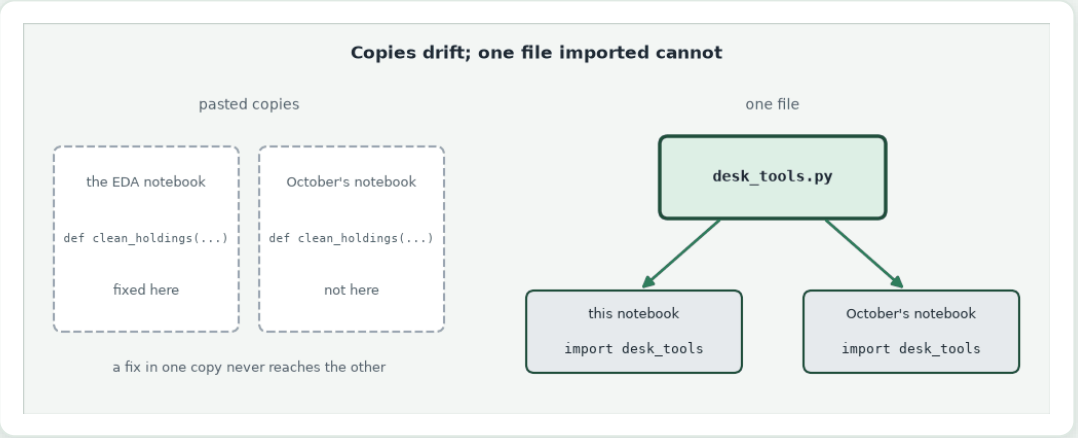
The module holds five functions and "pd", its own import of pandas. "desk_tools.load_holdings" is the same dot as "pd.read_csv": pandas is a module too.

A function in a module sees its parameters and the file's own names, never the notebook's, so the file has its own "import pandas as pd". A pasted copy drifts; one imported file cannot.

Five lines ran the EDA notebook's routine from a file and tied the cover note to the cent.

```
with open('desk_tools.py') as file:
    lines = file.readlines()
print(len(lines), 'lines. The lines that start a function:')
for line in lines:
    if line.startswith('def '):
        print(line.strip())
108 lines. The lines that start a function:
def load_holdings(path: str) -> pd.DataFrame:
def clean_holdings(raw):
def check_totals(book, bonds, face, market_value):
def summarize(book):
def write_report(book, summary, path):
```

The file is plain text, like Day 0's "half_coupon.py": open it in VS Code and it reads like the cell that wrote it.



19.2 Ask the module: help() and a type hint

```
help(desk_tools.clean_holdings)
Help on function clean_holdings in module desk_tools:

clean_holdings(raw)
    Apply the desk's cleaning decisions to an extract and return the cleaned table.

    The six decisions of the EDA notebook, in the same order:
      1. Exact duplicate rows are removed, keeping the first of each.
      2. Tickers are tidied: no spaces at either end, all capitals.
      3. A price dated on another day than the as-of date is flagged in a new column,
         stale_price. The price is not changed.
      4. A missing rating is labelled 'Not rated'. The bond stays in the book.
      5. A yield entered as a decimal is multiplied by 100. The test is the IQR rule:
         a yield below the first quartile less 1.5 times the IQR.
      6. Bonds with very wide spreads are real, and they are kept.

    raw is not changed. The cleaned table has one more column than raw: stale_price.
```

The signature, then the docstring word for word: the documentation next month's user reads, and that user is often you.

```
help(desk_tools.load_holdings)
Help on function load_holdings in module desk_tools:

load_holdings(path: str) -> pandas.DataFrame
    Read a holdings extract from a CSV file and return it as it arrived.

    path is the file name or the web address of the CSV. The CUSIP is read as text.
```

"path: str" and "-> pandas.DataFrame" are type hints: the type in after the colon, the type out after the arrow. Python does not enforce them; this one is the module's only hint, there so you can read them in other people's code.

The file documents itself: what goes in, what comes out, in which unit.

Excel side by side.

Job	Python	Excel
Steps every workbook can use	`import desk_tools`	macros in the Personal Macro Workbook, PERSONAL.XLSB, which Excel opens hidden at start-up
A named function you write	`def`	=LAMBDA(oas, oas / 100) named in Formulas, Name Manager, for that workbook only (Microsoft 365, Excel 2024)
A function's arguments	`help()`	type =SUM in a cell, then Ctrl+A: the Function Arguments box

19.3 Checks that stop the run

```
desk_tools.check_totals(raw, expected_bonds, expected_face, expected_value)
```

```
ValueError: control totals do not match: 204 rows against 200 bonds; face of
495,500,000 against 489,000,000 dollars; market value of 497,769,925.00
against 490,819,215.00 dollars
```

Read from the bottom, as on week 1's Day 3: one message, three mismatches. The lower arrow is the "raise" inside "desk_tools.py"; the upper is this notebook's call. A printed difference can be scrolled past; an error stops Run All before a report is written.

```
untidy = raw['ticker'] != raw['ticker'].str.strip().str.upper()
print(len(raw), raw.duplicated().sum(), untidy.sum(), raw['rating'].isna().sum())
204 4 8 5
```

Still 204, 4, 8, and 5: the function worked on a copy, so "raw" is as it arrived.

Excel side by side. CUSIP in column B, face in R, market value in S; every version.

Check	Excel
Bonds	=COUNTA(B2:B201)
Face	=SUM(R2:R201)
Market value	=SUM(S2:S201)
One is off	the EDA notebook's IF cell only shows CHECK; check_totals stops the run

The routine stops on a file that does not tie, and never changes the one it was given.

bonds	face	market_value	mean_oas	mean_yield	not_rated	stale_prices
200	489,000,000	490,819,215	185.75	6.088	5	6

The mean OAS is 185.75 bp and the mean yield 6.088 percent, as in the EDA notebook; 5 Not rated and 6 stale prices stay open for operations.

19.4 and 19.5 A sixth function, and a program

```
%%writefile -a desk_tools.py
def count_changes(raw, book):
    raw is the extract as it arrived and book is the table clean_holdings returned for it.
    kept = raw.drop_duplicates().reset_index(drop=True)
    counts = {
        'rows_removed': len(raw) - len(book),
        'tickers_tidied': (kept['ticker'] != book['ticker']).sum(),
        'stale_prices_flagged': book['stale_price'].sum(),
        'ratings_labelled': kept['rating'].isna().sum(),
        'yields_corrected': (kept['yield_pct'] != book['yield_pct']).sum(),
    }
    return pd.DataFrame([counts])
```

The summary shows the book, not what cleaning did: the desk wants a count per decision, a sixth function. "%%writefile -a" appends to the file, like week 1's Day 2 'a' mode.

Many ".py" files end with the block below. Add it to the end of "desk_tools.py".

```
%%writefile -a desk_tools.py
if __name__ == '__main__':
    print('desk_tools holds the month-end functions: import it to use them.')
```

```
%run desk_tools.py
desk_tools holds the month-end functions: import it to use them.
```

From a terminal, as Day 0 ran "half_coupon.py", the same is "python desk_tools.py".

19.6 The same lines on a second file

```
files = ['holdings_messy.csv', 'holdings.csv']
for file in files:
    received = desk_tools.load_holdings(data_folder + file)
    try:
        desk_tools.check_totals(received, expected_bonds, expected_face, expected_value)
        print(file, 'matches the cover note as received')
    except ValueError as error:
        print(file, error)
holdings_messy.csv control totals do not match: 204 rows against 200 bonds; face of 495,500,000 against 489,000,000 dollars; market value of 497,769,925.00 against 490,819,215.00 dollars
holdings.csv matches the cover note as received
```

Catch an error to report it, as here; the run below catches nothing, so a table that does not tie stops it.

```
books = {}
summaries = {}
changes = {}
for file in files:
    received = desk_tools.load_holdings(data_folder + file)
    books[file] = desk_tools.clean_holdings(received)
    desk_tools.check_totals(books[file], expected_bonds, expected_face, expected_value)
    summaries[file] = desk_tools.summarize(books[file])
    changes[file] = desk_tools.count_changes(received, books[file]).iloc[0]
```

The point of a module: the same lines on another file.

Excel side by side.

Job	Python	Excel
Next month's file	change the file name	re-point every range: S2:S205 on the 204-row extract, S2:S201 on the 200-row file
Same steps, new source	the loop above	Power Query: Data, Get Data, Data Source Settings, Change Source, then Refresh (Excel 2016 and later)

A range that stopped short, and the report workbook

Herndon, Ash and Pollin, 2013

In 2013 Thomas Herndon, Michael Ash and Robert Pollin checked the working spreadsheet behind Reinhart and Rogoff's 2010 paper "Growth in a Time of Debt" and found an average whose range stopped short, leaving five countries out: Australia, Austria, Belgium, Canada, and Denmark. A range does not move when the rows do.

Sources: PERI Working Paper 322, April 2013, https://peri.umass.edu/publication/does-high-public-debt-consistently-stifle-economic-growth-a-critique-of-reinhart-and-rogoff/; Cambridge Journal of Economics 38(2), 257-279, 2014, https://doi.org/10.1093/cje/bet075.

The PM reads Excel: "write_report" writes two sheets with Combining Part 2's "pd.ExcelWriter".

```
report_file = 'month_end_report.xlsx'
desk_tools.write_report(book, summary, report_file)
summary_back = pd.read_excel(report_file, sheet_name='summary')
book_back = pd.read_excel(report_file, sheet_name='book', dtype={'cusip': str})
desk_tools.check_totals(book_back, expected_bonds, expected_face, expected_value)
print(summary_back.shape, book_back.shape)
os.remove(report_file)
(1, 7) (200, 20)
```

"check_totals" raised nothing on the sheet read back: a function does not care where its table came from. The workbook holds values, not formulas.

The PM's workbook still ties after the round trip.

Excel side by side.

Job	pandas	Excel
Two tables, one file	pd.ExcelWriter, one to_excel(sheet_name=) each	one workbook, two sheets
Totals of the sheet	check_totals(book_back, ...)	=COUNTA(book!B2:B201), =SUM(book!R2:R201), =SUM(book!S2:S201)

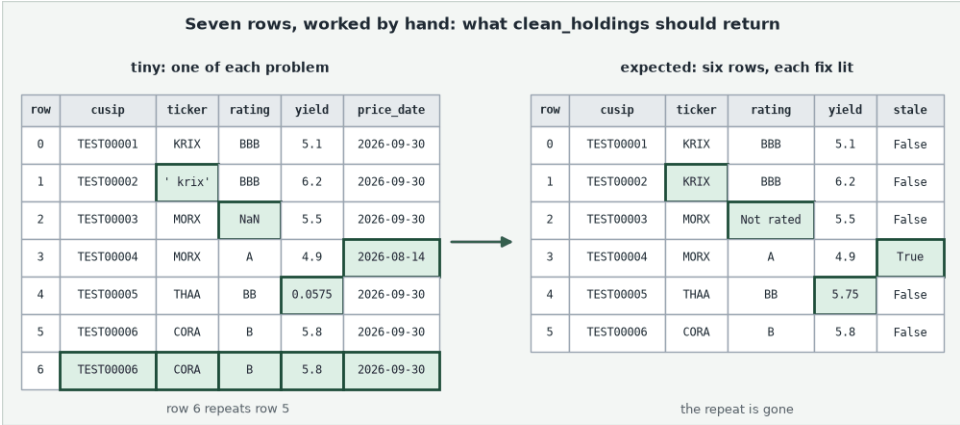
19.8 Testing a function on a table worked by hand

The totals test a run; a test checks the function itself, on a table small enough to work by hand. Break the ticker step next year and the totals still tie; a test on tickers fails.

```
tiny = pd.DataFrame({
    'cusip': ['TEST00001', 'TEST00002', 'TEST00003', 'TEST00004', 'TEST00005',
              'TEST00006', 'TEST00006'],
    'ticker': ['KRIX', ' krix', 'MORX', 'MORX', 'THAA', 'CORA', 'CORA'],
    'rating': ['BBB', 'BBB', None, 'A', 'BB', 'B', 'B'],
    'yield_pct': [5.1, 6.2, 5.5, 4.9, 0.0575, 5.8, 5.8],
    'as_of_date': ['2026-09-30'] * 7,
    'price_date': ['2026-09-30', '2026-09-30', '2026-09-30', '2026-08-14', '2026-09-30',
                   '2026-09-30', '2026-09-30'],
})
tiny
```

	cusip	ticker	rating	yield_pct	as_of_date	price_date
0	TEST00001	KRIX	BBB	5.1000	2026-09-30	2026-09-30
1	TEST00002	krix	BBB	6.2000	2026-09-30	2026-09-30
2	TEST00003	MORX	NaN	5.5000	2026-09-30	2026-09-30
3	TEST00004	MORX	A	4.9000	2026-09-30	2026-08-14
4	TEST00005	THAA	BB	0.0575	2026-09-30	2026-09-30
5	TEST00006	CORA	B	5.8000	2026-09-30	2026-09-30
6	TEST00006	CORA	B	5.8000	2026-09-30	2026-09-30

Worked by hand, the answer is six rows: " krix" tidied, row 2 Not rated, 0.0575 made 5.75, and row 3 flagged stale.



row	cusip	ticker	rating	yield	price_date
0	TEST00001	KRIX	BBB	5.1	2026-09-30
1	TEST00002	' krix'	BBB	6.2	2026-09-30
2	TEST00003	MORX	NaN	5.5	2026-09-30
3	TEST00004	MORX	A	4.9	2026-08-14
4	TEST00005	THAA	BB	0.0575	2026-09-30
5	TEST00006	CORA	B	5.8	2026-09-30
6	TEST00006	CORA	B	5.8	2026-09-30

row	cusip	ticker	rating	yield	stale
0	TEST00001	KRIX	BBB	5.1	False
1	TEST00002	KRIX	BBB	6.2	False
2	TEST00003	MORX	Not rated	5.5	False
3	TEST00004	MORX	A	4.9	True
4	TEST00005	THAA	BB	5.75	False
5	TEST00006	CORA	B	5.8	False

```
tidy = desk_tools.clean_holdings(tiny)
assert len(tidy) == 6, f'expected 6 rows, found {len(tidy)}'
assert tidy['cusip'].is_unique, 'a CUSIP is still repeated'
assert len(tiny) == 7, 'the table passed in was changed'
print('Test 1 passed:', len(tiny), 'rows in,', len(tidy), 'out')
Test 1 passed: 7 rows in, 6 out
```

Tests that pass, one that fails, and which jobs to move

```
expected_tickers = ['KRIX', 'KRIX', 'MORX', 'MORX', 'THAA', 'CORA']
assert tidy['ticker'].tolist() == expected_tickers, f"tickers are {tidy['ticker'].tolist()}"
print('Test 2 passed:', tidy['ticker'].tolist())
assert tidy['yield_pct'].tolist() == [5.1, 6.2, 5.5, 4.9, 5.75, 5.8], 'another yield moved'
print('Test 3 passed:', tidy['yield_pct'].tolist())
Test 2 passed: ['KRIX', 'KRIX', 'MORX', 'MORX', 'THAA', 'CORA']
Test 3 passed: [5.1, 6.2, 5.5, 4.9, 5.75, 5.8]
```

The yield test has two halves: fix the wrong yield and leave the right ones alone. A colleague remembers the label as "NR".

```
assert tidy.loc[2, 'rating'] == 'NR', f"expected the label NR, found {tidy.loc[2, 'rating']}"
print('Test 4 passed')
AssertionError: expected the label NR, found Not rated
```

The docstring settles it: decision 4's label is "Not rated", so the test is wrong.

```
def test_clean_holdings():
    result = desk_tools.clean_holdings(tiny)
    assert len(result) == 6, f'expected 6 rows, found {len(result)}'
    assert result['ticker'].tolist() == expected_tickers, 'tickers not tidied'
    assert result.loc[2, 'rating'] == 'Not rated', 'missing rating not labelled'
    assert result['yield_pct'].tolist() == [5.1, 6.2, 5.5, 4.9, 5.75, 5.8], 'yields wrong'
    assert result['stale_price'].tolist() == [False, False, False, True, False, False], 'stale'
    print('clean_holdings: 5 checks passed')
test_clean_holdings()
clean_holdings: 5 checks passed
```

Excel side by side

A sheet of made-up rows with known answers, where a formula is tried before it meets 200 real rows.

Run it after every reload. "pytest", a tool you will meet, runs every function whose name starts with "test_", with one command.

A test fails loudly where the totals stay silent.

Move a job when it shows the signs:

Sign	September's routine
It repeats	every month end
The input changes shape	204 rows, then 200
It has control totals	the cover note's three
Others rely on it	the PM's workbook
The steps are hard to see	six decisions, now in one docstring

Predict first: decide, then run each in the notebook

Predict 1

The file now holds "count_changes"; the notebook imported "desk_tools" in 19.1. What does "desk_tools.count_changes(raw, book)" do? A) it works: the function is in the file B) an error: the notebook uses the module as it was imported C) a "NameError": "count_changes" is not defined

```
desk_tools.count_changes(raw, book)
```

Predict 2

Reload the module with the block in it. What does the reload print? A) the block's line: a reload runs the whole file B) nothing C) a "NameError": nothing defines "__name__"

```
importlib.reload(desk_tools)
print(desk_tools.__name__)
```

Predict 3

Both files go through the same lines. Add the five "count_changes" counts for "holdings.csv": what total? Write down a number; some expect the IQR rule to move a yield or two.

```
for file in files:
    received = desk_tools.load_holdings(data_folder + file)
    books[file] = desk_tools.clean_holdings(received)
    desk_tools.check_totals(books[file], expected_bonds, expected_face, expected_value)
    summaries[file] = desk_tools.summarize(books[file])
    changes[file] = desk_tools.count_changes(received, books[file]).iloc[0]
```

The note, answered

The note asked	Answer
One file	desk_tools.py, six functions, imported
Tie the cover note	200 bonds, 490,819,215.00 dollars
Count the changes	4, 8, 6, 5, and 2
The workbook	two sheets, tied after the round trip
Test the function	five checks pass; a wrong test fails loudly
Which jobs next	the five signs

Move a job when it repeats, changes shape, has control totals, feeds others, and hides its steps.

Next is the Course 1 Project: a new extract, a new cover note, and no list of what is wrong.